



INDEX

Ex. No.	Name of the Exercise	Page No.
UNIX COMMANDS		
1	STUDY OF UNIX OPERATING SYSTEM	1
2	STUDY OF BASIC SHELL COMMANDS	5
3	STUDY OF FILTERS AND SEARCHING	8
4	STUDY OF VI EDITOR	10
SHELL PROGRAMMING		
5	SIMPLE SHELL PROGRAM	13
6	CONDITIONAL STATEMENTS	16
7	MULTI-WAY BRANCHING	19
8	TESTING AND LOOPS	23
C PROGRAMMING ON UNIX		
9	FUNCTIONS	26
10	FILE HANDLING	28



EX.NO:1

STUDY OF UNIX OPERATING SYSTEM

Date:

Objective: To learn the fundamentals of UNIX Operating System.

Outcome: Acquire knowledge about UNIX Operating System and its various aspects.

Pre-request/Theme: Students should know the basic concepts of a Computer operating system.

Description:

OPERATING SYSTEM

An operating system (commonly abbreviated OS and O/S) is the software component of a computer system that is responsible for the management and coordination of activities and the sharing of the limited resources of the computer. It is a set of programs that provides the interface between user and computer hardware. Controlled allocation of the processors, memories, and I/O devices among the programs are allocated by the operating systems efficiently. So the Operating System is also known as resource allocator. Operating systems offer a number of services to application programs and users. Applications access these services through application programming interfaces (APIs) or system calls.

TYPES OF OPERATING SYSTEM

1. Single User - The system will have its own hard disk, memory, CPU and other resources all dedicated to a single user. E.g. MS-DOS

2. Multi User- The users have access to a multi-user system will have just a terminal and a keyboard. The other resources such as hard disk, printers are centrally located. The user is expected to simply hook onto his account, perform the work, disconnect and leave quietly.

E.g. UNIX

UNIX HISTORY

The spade work for UNIX began at AT&T Bell Laboratories in 1969 by Ken Thompson and Dennis Ritchie. The OS was initially known as UNICS (UNiplexed Information and Computing System). In 1970 UNICS finally became UNIX. In 1973, UNIX was rewritten in C and authored. UNIX operating systems are widely used in both servers and workstations.

UNIX was designed to be portable, multi-tasking and multi-user in a timesharing configuration. UNIX systems are characterized by various concepts: the use of plain text for storing data; a hierarchical file system; treating devices and certain types of inter-process communication (IPC) as files; and the use of a large number of software tools, small programs that can be combined together through a command line interpreter. These concepts are known as the UNIX philosophy.



UNIX FEATURES

- 1. Multi-user system**—Multi-user capability of UNIX allows several users to use the same computer to perform their tasks. Several terminals [Keyboards and Monitors] are connected to a single powerful computer [UNIX server] and each user can work with their terminals.
- 2. Multi-tasking system**—Multitasking is the capability of the operating system to perform various task simultaneously, i.e. a user can run multiple tasks concurrently.
- 3. Programming Facility**—UNIX is highly programmable, the UNIX shell has all the necessary ingredients like conditional and control structures, etc.
- 4. Security**—UNIX allows sharing of data; every user must have a single login name and password. So accessing another user's data is impossible without his permission.
- 5. Portability**—UNIX is portable because it is written in a high level language. So, UNIX can be run on different computers.
- 6. Communication**—UNIX supports communication between different terminals of the same server as well as between terminals on different servers. Apart from these features, UNIX has an extensive Tool kit, exhaustive system calls and Libraries and enhanced GUI (X Window).

ORGANIZATION OF UNIX

The UNIX system is functionally organized at three levels :

1. The kernel, which schedules tasks and manages storage.
2. The shell (Command Interpreter), which connects and interprets users' commands, calls programs from memory, and executes them.
3. The tools and applications that offer additional functionality to the OS.

COMPONENTS OF UNIX OPERATING SYSTEM

Kernel

The kernel is the heart of the system, a collection of programs written in C that directly communicate with the hardware. It manages the system resources, allocates time between user and processes, decides process priorities, and performs all other tasks. It's that part of UNIX system that is loaded into memory when the system is booted. So we can define kernel as the Master program or often called the Operating system.

Shell

The user cannot directly interact with the kernel. During the login of the user, the kernel starts an interactive program for each user. This program is known as shell. It is actually the interface between the user and the kernel. When user gives some command the shell analyses those commands and passes them to the kernel. The kernel then submits these commands to hardware and the required actions are carried out.

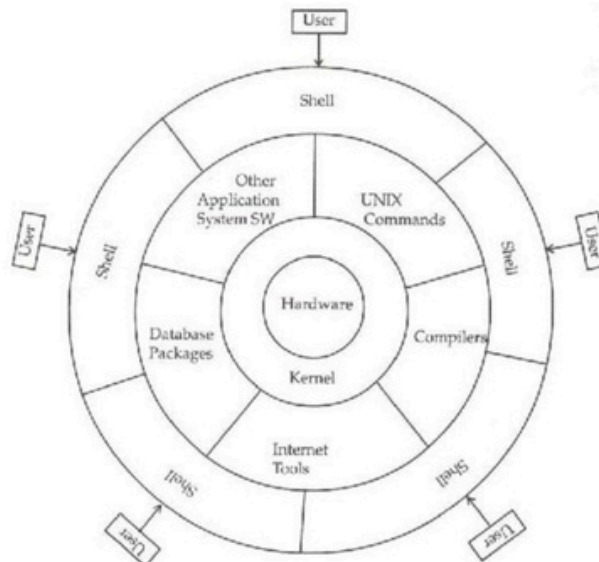


Fig 1.1: Structure of UNIX Operating System

FILES AND DIRECTORIES

A file is a collection of information that is assigned a name that is used to identify that file. The file is always stored in secondary storage medium. A directory is a special type file that contains a list of file names. All files are grouped together into directory for easier access. A directory can have one or more directories in it called sub-directories. In LINUX, files and directories are arranged in a hierarchical manner.

The files can be broadly classified as follows:

Ordinary files- Contains stream of data. All data, text, source programs, object and executable code, commands fall into this category.

Directory files- Contains no external data. It contains an entry, name of the file and its identification number for each file and subdirectory under that directory. Directory files are not created by the user but by the UNIX system itself.

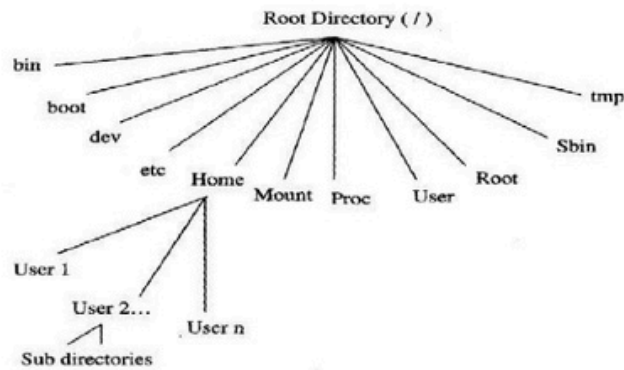
Device files - Even physical devices are treated as files. These are special in the sense that any output directed to it will be reflected onto the respective device.

UNIX FILE SYSTEM

All files in UNIX are related to one another. The file system of UNIX resembles a tree that grows from top to bottom. The file system begins with a directory called root (at the top). The root directory is denoted by a slash (/). Branching from root there are several directories such as bin, lib, etc, tmp, dev. Each of these directories contains several sub-directories and files.



Fig 1.2: UNIX File System



The details of the different types of the Directories are given below:

Directory	Contents of Directories
/ bin	executable files
/ boot	Linux kernel
/ dev	device files
/ etc	administrative files
/ home	User area
/ lib	C compiler and subroutines
/ mnt	Mount information of floppy and CD
/ proc	process information files
/ root	home area
/ sbin	configuration and system running files
/ tmp	temporary files
/ usr	user file system
/ var	logs, spools for printing and other files.

CONCLUSION:

Thus the study of Unix Operating System has been completed successfully.



EX.NO:2

STUDY OF BASIC SHELL COMMANDS

Date:

Objective: To study and execute the shell commands.

Outcome: work with basic shell commands.

Pre-request/Theme: Students should know the basic commands of DOS.

Description:

UNIX is security conscious, and can be used only by those persons who have an account. *Telnet* (Telephone Network) is a Terminal emulator program for TCP/IP networks that enables users to log on to remote servers.

To *logon*, type `telnet server_ipaddress` in run window.

User has to authenticate himself by providing *username* and *password*. Once verified, a greeting and \$ prompt appears. The shell is now ready to receive commands from the user. Options suffixed with a hyphen (-) and arguments are separated by space.

UNIX Shell:

A UNIX shell is a command-line interpreter that provides a user interface for the UNIX operating system. Users direct the operation of the computer by entering commands as text for a command line interpreter to execute or by creating text scripts of one or more such commands.

The commands can be combined using the pipeline (|) operator.

For example, number of users logged in can be obtained as → `who | wc -l`

COMMAND SHELL: It is a program that interprets commands. It allows a user to execute commands by typing them manually at a terminal, or automatically in programs called shell scripts. A shell is not an operating system. It is a way to interface with the operating system and run commands.

BASH = Bourne Again Shell.

Bash is a shell written as a free replacement to the standard Bourne Shell (/bin/sh) originally written by Steve Bourne for UNIX systems. It has all of the features of the original Bourne Shell, plus additions that make it easier to program with and use from the command line. Since it is Free Software, it has been adopted as the default shell on most Linux systems.



General commands:

Command	Function
date	Used to display the current system date and time.
date + %D	Displays date only
date + %T	Displays time only
date + %Y	Displays the year part of date
date +%H	Displays the hour part of time
cal	Calendar of the current month
cal year	Displays calendar for all months of the specified year
cal month year	Displays calendar for the specified month of the year
who	Get the information about all the users currently working in the system
who am i	It is used to know in which terminal the user is currently logged on
id	It is used to display the login name.
Tty	Used to display the terminal name
uname	Displays the Operating System
uname-r	Shows version number of the OS (kernel).
uname-n	Displays domain name of the server
echo "txt"	Displays the given text on the screen
echo \$HOME	Displays the user's home directory
Bc	Basic calculator. Press Ctrl+d to quit
Lp file	Allows the user to spool a job along with others in a print queue.
man cmdname	Manual for the given command. Press q to exit
history	To display the commands used by the user since logon.
Exit	Exit from a process. If shell is the only process then logs out

Directory commands:

Command	Function
Pwd	Path of the present working directory
mkdir dir	A directory is created in the given name under the current directory
mkdir dir1dir2	A number of sub-directories can be created under one stroke
cd subdir	Change Directory. If the <i>subdir</i> starts with/then path starts from root (absolute)otherwise from current working directory.
cd	To switch to the home directory.
cd/	To switch to the root directory.
cd ..	To move back to the parent directory
rmdir subdir	Removes an empty sub-directory.



File commands:

Command	Function
cat >filename	To create a file with some contents. To end typing press Ctrl+d . The >symbol means redirecting output to a file.(< for input)
cat filename	Displays the file contents.
cat >>filename	Used to append contents to a file
cp src des	Copy files to given location. If already exists, it will be overwritten
cp-isrc des	Warns the user prior to overwriting the destination file
cp-rsrc des	Copies the entire directory, all its sub-directories and files.
mv old new	To rename an existing file or directory. -i option can also be used
mv f1 f2 f3 dir	To move a group of files to a directory.
mv-v old new	Display name of each file as it is moved.
file filename	Determine the type of file
rm file	Used to delete a file or group of files. -i option can also be used
spell filename	Find the spelling errors in the file
rm*	To delete all the files in the directory.
rm-r*	Deletes all files and sub-directories
rm-f*	To forcibly remove even write-protected files
ls	Lists all files and subdirectories (blue colored) in sorted manner.
ls name	To check whether a file or directory exists.
ls name*	Short-hand notation to list out file names of a specific pattern.
ls-a	Lists all files including hidden files (files beginning with.)
ls-x dir name	To have specific listing of a directory.
ls-R	Recursive listing of all files in the subdirectories
ls-l	Long listing showing file access rights (read/write/execute- rw x for user/group/others- ugo).
cmp file1file2	Used to compare two files. Displays nothing if files are identical.
wc file	It produces a statistics of lines (l), words (w), and characters(c).
chmod perm file	Changes permission for the specified file.(r=4,w=2,x=1) chmod740filesetsallrightsforuser,readonlyforgroups and no rights for others

CONCLUSION:

Thus the study and execution of basic shell commands has been completed successfully.



EX.NO:3

STUDY OF FILTERS AND SEARCHING

Date:

Objectives: a) To query a data file using filter commands in UNIX

b) To search for a regular expression in a file using grep command in UNIX.

Outcome: solve problems using filtering commands and grep commands in UNIX

Pre-request/Theme: Students should know about File structure and File commands in UNIX.

Description:

Filters are the central commands of the UNIX tool kit. It acts on data file where lines are *records*, *fields* delimited by a character not used by the data (mostly |, default is white space). The output is a set of records and the input file is unaltered by these commands.

Command	Function
head	used to display the first few records (10 records by default)
head stud	Displays first 10 records by default
head -5 stud	Displays first 5 records
head -l stud wc -c	length of first record
tail	used to display the last few records (10 records by default)
tail stud	Displays last 10 records by default
tail -5 stud tee last5	Last 5 records listed & stored in file <i>last5</i> using <i>tee</i>
cut	Used to extract specific fields. The d option specifies the delimiter and f for specifying the field list. The c option may be used if extraction is done character wise
cut -d \ -f 1,3,4	Fields 1,3,4 listed
stud paste -d \ list1	merges two cut files <i>list1</i> and <i>list2</i>
list2 sort	Reorders the file as per ASCII sequence. The t option is used to specify delimiter and k option to specify the field
sort stud	Sorted on 1st column by default
sort -t \ -k 3 stud	Sort as per 3rd column
sort -c stud	Check if file is sorted using c option
sort -t \ -k 4,4 -k 3,3 stud	Sorting on secondary keys
sort -t \ -nr -k 4 stud	Sort on numeric field using n option , r for reverse
uniq stud	Display unique entries in a sorted file
tr	Translates characters. Can be used to change text case. It works with standard input <
tr 'a-z' 'A-Z' <	Changes text to upper case
stud nl	Display file content with lines numbered. The s option is used to specify separator
nl -s " " stud	Displays entries numbered with separator
pr [options] <file name >	It is used to display the contents of the file by separating them into
join -a1 f1 f2	It is used to extracts common lines from two sorted files

A frequent requirement is to look for a pattern or expression in a file. UNIX handles this feature through **grep** and **egrep**. **grep** uses an regular expression to display lines that match and **egrep** enables searching for multiple patterns. Its usage is `grep options searchtext filename`



Command	Function
grep this demo	Lists the lines that contains the string <i>this</i>
grep 'end of' demo	Quotes mandatory for text containing space
grep this demo*	Search <i>this</i> in multiple files
grep -c to demo	Number of occurrence of the word <i>to</i> in the file
grep -n sequence demo	Display line numbers along with matching lines
grep -v word demo	Displays lines that does not contain the text
grep -l vim *	<i>word</i> Displays files containing text <i>vim</i>
grep -i WORD demo	Search for text ignoring case differences
grep '^[0-9]' demo	Lines that start with a number
grep '[0-9]\$_' demo	Lines that end with a number
ls -l grep "^d"	Display the subdirectory names
grep -c "^\$" demo	Display count of blank lines in the file.
grep "2....\$" stud	Display lines that ends in the range 20000–
egrep "lower UPPER" demo	29999 Display lines that match either <i>lower</i> or
egrep "(previous current) word" demo	<i>upper</i> Display lines that match either <i>previous</i> word or <i>current word</i>

CONCLUSION:

Thus information retrieval using filters has been completed successfully

Thus searching text patterns in files using grep has been completed successfully.



EX.NO:4

STUDY OF VI EDITOR

Date:

Objective: To learn the various features and controls of VI editor.

Outcome: Acquire knowledge about editing text using VI editor and execute UNIX commands.

Pre-requisite: Students should know the various types of commands in UNIX.

vi Editor:

A *text editor* is one of the most common and useful tools in all Operating Systems. Unix provides a versatile editor vi, a full-screen editor and owes its origin to Bill Joy. "vi" stands for *visual* editor. A vi session begins by invoking vi with or without a filename

\$vi

\$vifilename

The user is presented with a full empty screen, each line beginning with a ~. This is vi's way of indicating non-existent lines. Out of 25 lines on the terminal, 24 can be used to enter text. The last line is reserved for commands and also used by the system to display messages. vi functions in three modes namely:

1. **Input mode**—where any key depressed is entered as text
2. **Command mode**—where keys are used as commands to act on text (initial mode)
3. **ex mode**—ex mode commands that can be entered in the last line to act on text

INPUT MODE:

vi starts with command mode. To insert text any of the following commands should be used.

Commands	Function
i	Inserts text to the left of the cursor.
I	Inserts text at the beginning of line.
a	Appends text to right of cursor
A	Appends text at end of line
o	Opens line below
O	Opens line above

In *Input* mode the editor displays **INSERT** in the last line. Press *Enter* key to start a fresh line of text in *Input* mode and the ~ disappears. To quit *input* mode press *Esc* key.



COMMAND MODE:

EDIT COMMANDS

Command	Function
R	Replaces more than a single character. The editor displays REPLACE in the last line. The existing text is overwritten as they are typed.
S	Deletes a single character to the left and switches to Input
X	Deletes the character in the current cursor position
?text	Locates the <i>text</i> in the file. If not found, the message "Pattern not found" appears. Use n to repeat the forward search and N for backward search.
U or u	Reverses the last change made to the buffer.
Dd	Cuts the entire line
Dw	Cuts the entire word
d\$	Cuts a line from cursor position to the end of the line
d0	Cuts from the cursor position to the start of the line
Yy	Copies (yanks) the entire line
Yw	Copies the entire word
P	Pastes the text

NAVIGATION COMMANDS

Command	Function
B	Moves back to beginning of a word
W	Moves forward to beginning of word
	Moves to start of the line
\$	Moves to end of the line
K	Up one line
J	Down one line
H	Left one character
L	Right one character
Ctrl+f	Scrolls a page forward
Ctrl+b	Scrolls a page backward
/G	To move to the specific line

One of the most notable features of **vi** is the facility of prefixing a number to most commands. When prefixed, commands interpret the instruction to be repeated as many times. For example **3x** deletes the next three characters.



THE EX MODE:

The essential save and exit commands form the features of ex mode. Press : (colon) in command mode to switch to ex mode. The : is displayed in the last line. Type the command and press *Enter* key to execute the same.

Command	Function
W	Saves file, displays the number of lines & characters and returns to Input mode. If it is an unnamed file then vi puts a message.
w <i>file</i> L1,L2	The file is saved under the given name
w <i>file</i>	Used to write specific line numbers to some file. The . (dot) represents current line, 0 for first line and \$ could be used to represent last line.
Q	Quits vi session and returns to \$ prompt. vi has a safety mechanism that warns if changes made to file are not saved.
q!	Quits vi session without saving any changes made since the last
Wq	Save and exit
Sh	Escape to shell
%s/Sstr/Rstr/g	This is yet another powerful feature known as substitution. It is similar to Find and Replace. % represents all lines, g makes it global. To make vi ask for confirmation before replacing use gc instead of g.
r <i>file</i>	To insert another file into the current file.
new <i>file</i>	Splits screen into multiple windows and opens the file.

Starting with VI editor:

Syntax:vi filename

Editing the file:

- ◆ Open the file using \$ vi filename
- ◆ To add text at the end of the file, position the cursor at the last character of the file.
- ◆ Switch from command mode to text input mode by pressing 'a'.
- ◆ Here 'a' stands for append.
- ◆ Inserting text in the middle of the file is possible by pressing 'i'. The editor accepts and inserts the typed character until Esc key is pressed.

Deleting Text

For deleting a character, move the cursor to the character, press 'x'. The character will disappear.

CONCLUSION:

Thus the study of text manipulation using vi editor has been completed successfully.



EX.NO:5

SHELL PROGRAMMING – SIMPLE SHELL PROGRAM

Date:

Objective: Learn to write a simple shell program.

Outcome: Realize how to solve simple problems using shell script.

Pre-requisite: Students should know the basic commands of shell programming.

Description:

The original UNIX came with the Bourne shell (sh) and it is universal even today. Then came a plethora of shells offering new features. Two of them, C shell (csh) and Korn shell (ksh) has been well accepted by the UNIX fraternity. Linux offers Bash shell (bash) as a superior alternative to Bourne shell.

UNIX system will usually offer a variety of shell types:

- **sh** or Bourne Shell: The original basic shell, a small program with few features still used on UNIX systems and in UNIX-related environments
- **Bash** or Bourne Again shell: The standard GNU shell, intuitive and flexible. Probably most advisable for beginning users while being at the same time a powerful tool for the advanced and professional user. On Linux, **bash** is the standard shell for common users. This shell is a so-called *superset* of the Bourne shell, a set of add-ons and plug-ins.
- **csh** or C shell: The syntax of this shell resembles that of the C programming language.
- **tcsh** or TENEX C shell: a superset of the common C shell, enhancing user-friendliness and speed. That is why some also call it the Turbo C shell.
- **ksh** or the Korn shell: sometimes appreciated by people with a UNIX background. A superset of the Bourne shell; with standard configuration a nightmare for beginning users.

The activities of a shell are not restricted to command interpretation alone. The shell also has rudimentary programming features. When a group of commands has to be executed regularly, they are stored in a file (with extension .sh). All such files are called shell scripts or shell programs. Shell programs run in interpretive mode.

Key factors:

1. Comments in shell script start with #. It can be placed anywhere in a line; the shell ignores contents to its right. Comments are recommended but not mandatory
2. Shell variables are loosely typed i.e. not declared. Their type depends on the value assigned.
Variables when used in an expression or output must be prefixed by \$.



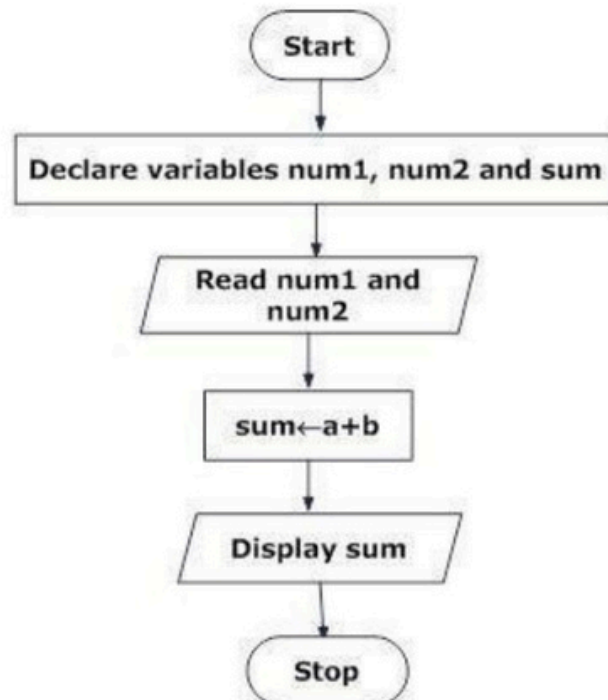
3. The read statement is shell's internal tool for making scripts interactive.
4. Output is displayed using echo statement. Any text should be within quotes. Escape sequence should be used with -e option.
5. Commands are always enclosed with `` (back quotes).
6. Expressions are computed using the expr command. Arithmetic operators are + - * /. Meta characters * () should be escaped with a \.
7. Multiple statements can be written in a single line separated by ;
8. The shell scripts are executed using the sh command (sh filename).

Sample program title: ADDITION OF TWO NUMBERS

ALGORITHM:

- Step 1- Open vi editor and enter the program
- Step 2- Read two numbers from the user.
- Step 3- Add both the numbers.
- Step 4- Print the summation of two numbers.
- Step 5- Stop.

FLOWCHART:





SHELL SCRIPT:

```
echo -n "Enter the 1st number"  
read n1  
echo -n "Enter the 2nd number"  
read n2  
sum = `expr $n1 + $n2`  
echo "The sum is = $sum"
```

INPUT	OUTPUT
\$ Enter the 1 st number : 23 Enter the 2 nd number : 45	The sum is = 68

CONCLUSION:

Thus the shell script to compute the summation of the two numbers was executed successfully.



EX.NO:6

SHELL PROGRAMMING – CONDITIONAL STATEMENTS

Date:

Objective: Recognize the usage of Conditional Statements in Shell Programming.

Outcome: Get to know how to solve problems involving certain conditions.

Pre-request/Theme: Students should know to solve basic problems using Shell scripts.

Description:

Shell supports decision-making using if statement. The if statement has the following formats. The first construct executes the *statements* when the condition is true. The second construct adds an optional else to the first one that has different set of statements to be executed depending on whether the condition is true or false. The last one is an elif ladder, in which conditions are tested in sequence, but only one set of statements is executed.

Type I : if [*condition*]
 then
 statements
 fi

Type II : if [*condition*]
 then
 statements
 else
 statements
 fi

Type III : if [*condition*]
 then
 statements
 elif [*condition*]
 then
 statements
 ..
 else
 statements
 fi

The set of relational and logical operators used in conditional expression is given below.

The numeric comparison in the shell is confined to integer values only.



Operator Description

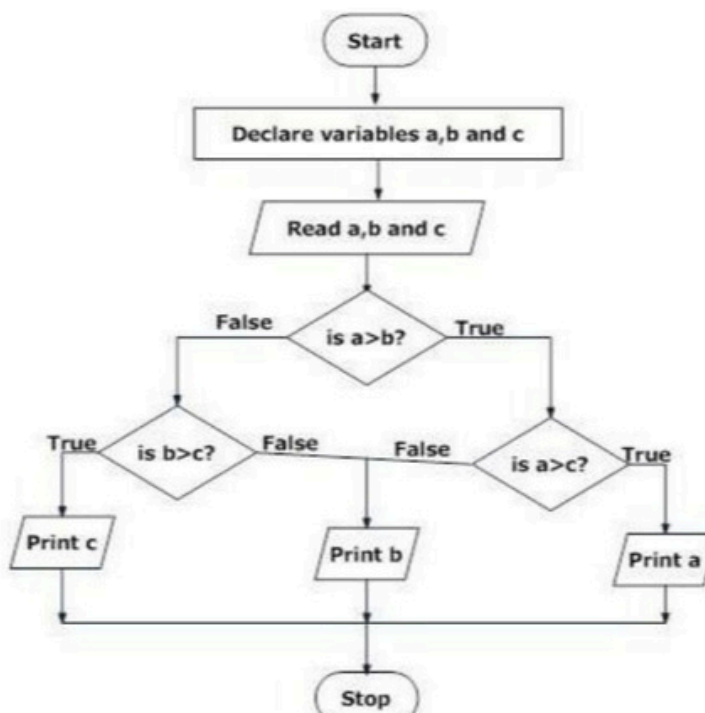
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-ge	Greater than or equal to
-lt	Less than
-le	Less than or equal to
-a	Logical AND
-o	Logical OR
!	Logical NOT

Sample program title: **BIGGEST AMONG THREE NUMBERS**

ALGORITHM:

- Step 1- Open vi editor and start the program
- Step 2- Read values of a, b and c
- Step 3- If $a > b$ and $a > c$ then
Print "A is the biggest" else if $b > c$ then
Print "B is the biggest"
else
Print "C is the biggest"
- Step 4- Stop

FLOWCHART:





SHELL SCRIPT

```
echo -n "Give value for A B and C: " read a b c
if [ $a -gt $b -a $a -gt $c ]
then
echo "A is the Biggest number" elif [ $b -gt $c ]
then
echo "B is the Biggest number" else
echo "C is the Biggest number"
fi
```

INPUT	OUTPUT
[vijai@localhost decision]\$ sh big3.sh Give value for A B and C: 4 3 4	C is the Biggest number

CONCLUSION:

Thus the shell script to find the biggest among three numbers was executed.



EX.NO:7

SHELL PROGRAMMING – MULTI WAY BRANCHING

Date:

Objective: To introduce the concept of Multi-way Branching in shell scripts.

Outcome: Know how to solve problems using Multi-way Branching statements.

Pre-request/Theme: Students should know about conditional Shell scripts.

Description:

The case statement is used to compare a variables value against a set of constants (integer, character, string, range). If it matches a constant, then the set of statements followed after) is executed till a ;; is encountered. The optional *default* block is indicated by *. Multiple constants can be specified in a single pattern separated by |.

Syntax: *case variable in*
 constant1)
 statements;;
 constant2)
 statements;;
 ...
 constantN)
 statements;;
 **)*
 statements;;
 esac

Sample program title: SIMPLE CALCULATOR

ALGORITHM:

- Step 1- Open vi editor and start the program
- Step 2- Read operands a and b
- Step 3- Display operation menu
- Step 4- Read option
 - Step 4.1- If option = 1 then
 - Step 4.2- Calculate $c = a + b$
 - Step 4.3- else if option = 2 then Calculate $c = a - b$
 - Step 4.4- else if option = 3 then
 - Step 4.5- Calculate $c = a * b$
 - Step 4.6- else if option = 4 then Calculate $c = a / b$
 - Step 4.7- else if option = 5 then
 - Step 4.8- Calculate $c = a \% b$

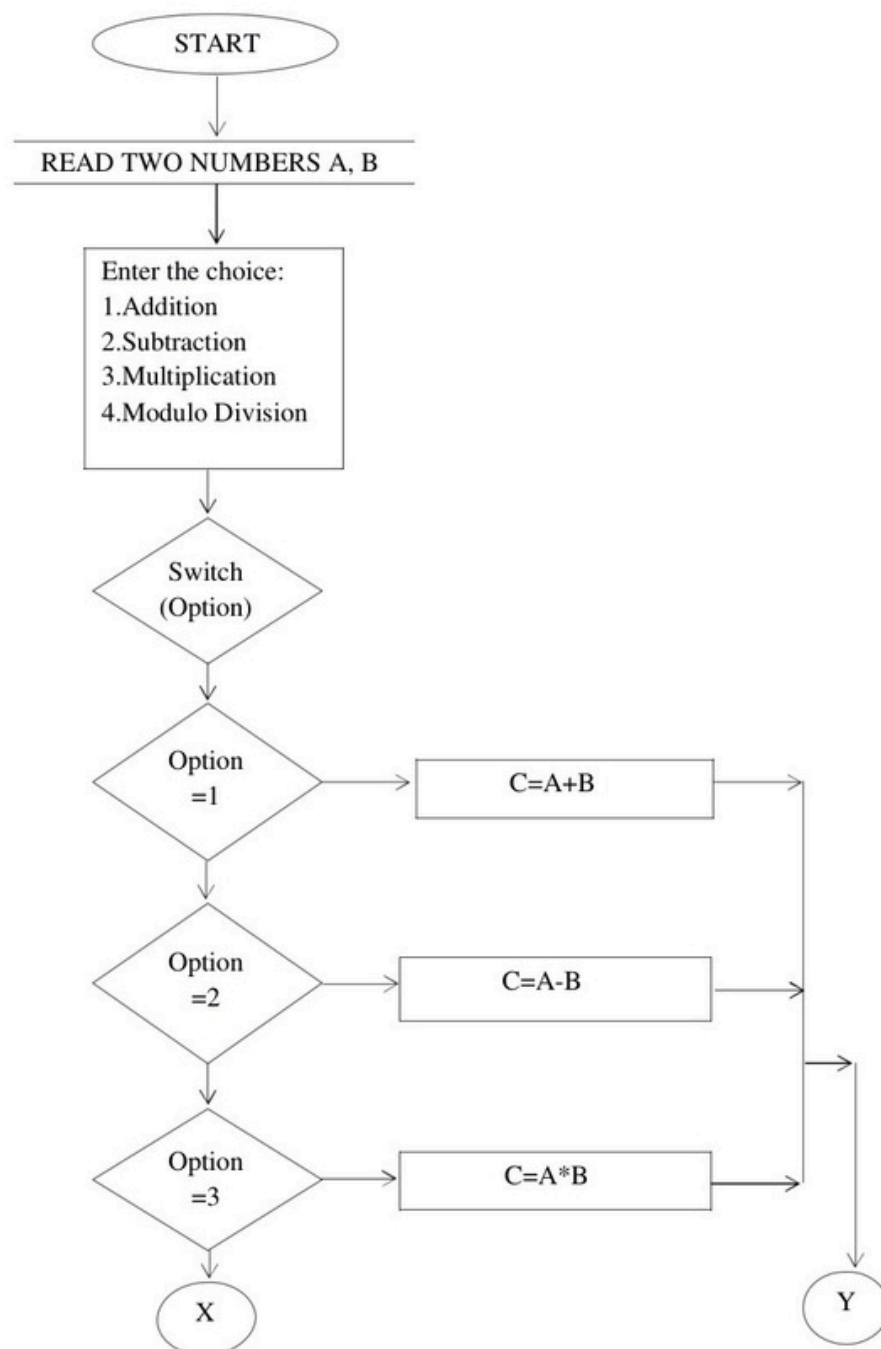


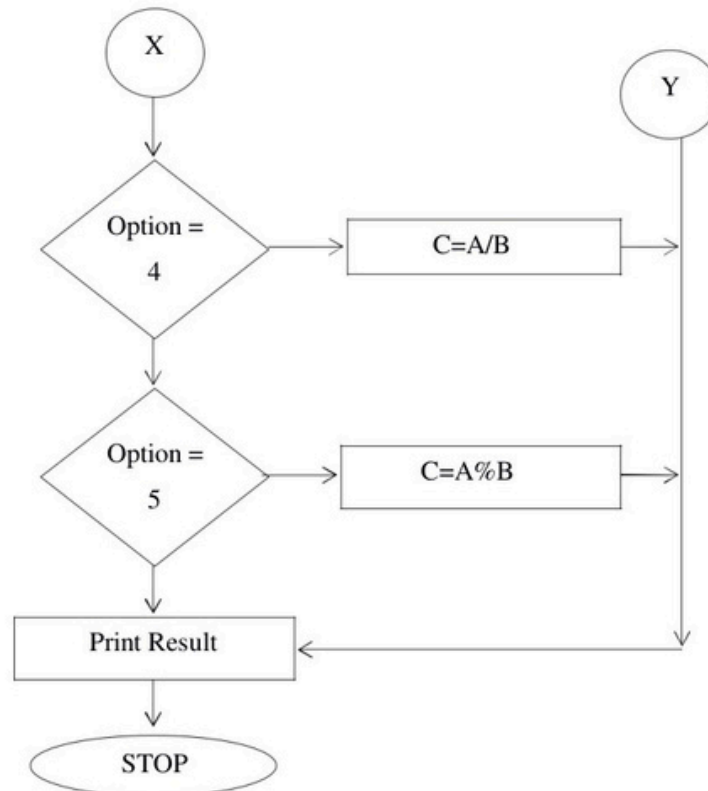
Step 4.9- elsePrint "Invalid option"

Step 5- Print c

Step 6- Stop

FLOWCHART:





SHELL SCRIPT

```
echo -n "Enter the two numbers : "
read a, b
echo " 1. Addition"
echo " 2. Subtraction"
echo " 3. Multiplication"
echo " 4. Division"
echo " 5. Modulo Division"
echo -n "Enter the option : "
read option
case $option in
    1) c=`expr $a + $b`
    echo "$a + $b = $c";;
    2) c=`expr $a - $b`
    echo "$a - $b = $c";;
    3) c=`expr $a \* $b`
    echo "$a * $b = $c";;
```



```
4) c=`expr $a / $b`  
echo "$a / $b = $c";;  
5) c=`expr $a % $b`  
echo "$a % $b = $c";;  
*) echo "Invalid Option"  
esac
```

INPUT	OUTPUT
[vijai@localhostmultway]\$ sh calc.sh Enter the two numbers : 2 4 1. Addition 2. Subtraction 3. Multiplication 4. Division 5. Modulo Division Enter the option : 1	2+4=6

CONCLUSION:

Thus the shell script to perform arithmetic operations using simple calculator was executed.



EX.NO:8

SHELL PROGRAMMING – TESTING AND LOOPS

Date :

Objective: Realize the Looping constructs in Shell programming.

Outcome: Know how to solve problems requiring testing and repetitions of statements.

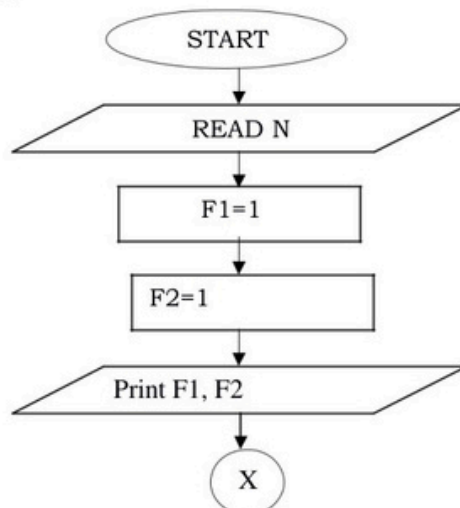
Pre-request/Theme: Students should know about Decision Making and Conditional manipulations.

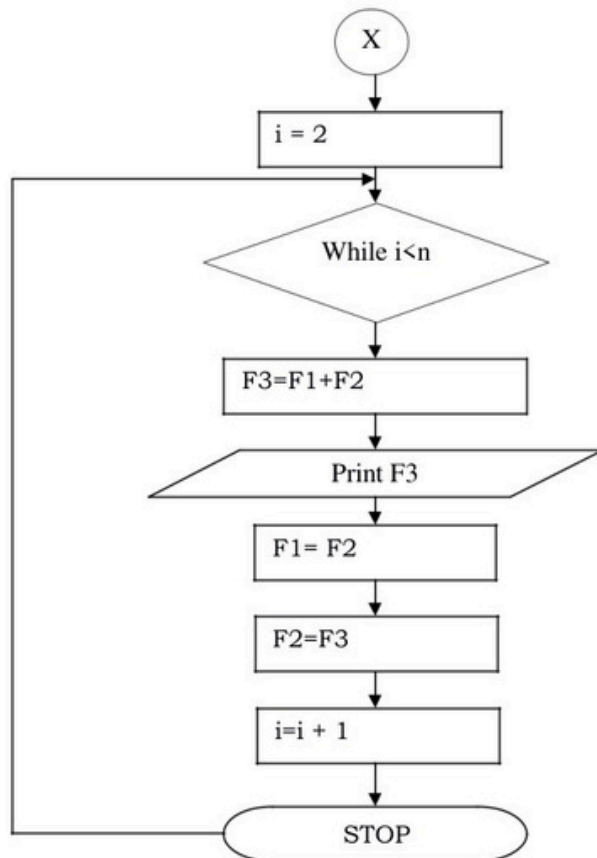
Sample program title: FIBONACCI SERIES

ALGORITHM:

- Step 1- Open vi editor and start the program
- Step 2- Read number of terms as n
- Step 3- Initialize 0 to f1, 1 to f2 and 2 to i
- Step 4- Print initial Fibonacci terms f1, f2
- Step 5- Generate next term using the formula $f3 = f1 + f2$
- Step 6- Print f3
- Step 7- Increment i by 1
- Step 8- Assign f2 to f1
- Step 9- Assign f3 to f2
- Step 10- Repeat steps 5–9 until i less than n
- Step 11- Stop

FLOWCHART:





SHELL SCRIPT:

```
echo -n "Enter number of terms : "
read n
echo "Fibonacci Series"
f1=0
f2=1
echo -n "$f1 "
echo -n "$f2 "
i=2
while [ $i -lt $n ]
do
    f3=`expr $f1 + $f2`
    echo -n "$f3 "
    f1=$f2
    f2=$f3
    i=`expr $i + 1`
done
```



INPUT	OUTPUT
[vijai@localhost loops]\$ sh fibo.sh Enter number of terms : 8	Fibonacci Series 011235813

CONCLUSION:

Thus the shell script to generate the Fibonacci series was successfully executed.



EX.NO:9

C PROGRAMMING ON UNIX – FUNCTIONS

Date :

Objective: To solve problems with the idea of functions using C program on UNIX.

Outcome: Learn to implement C Functions on UNIX.

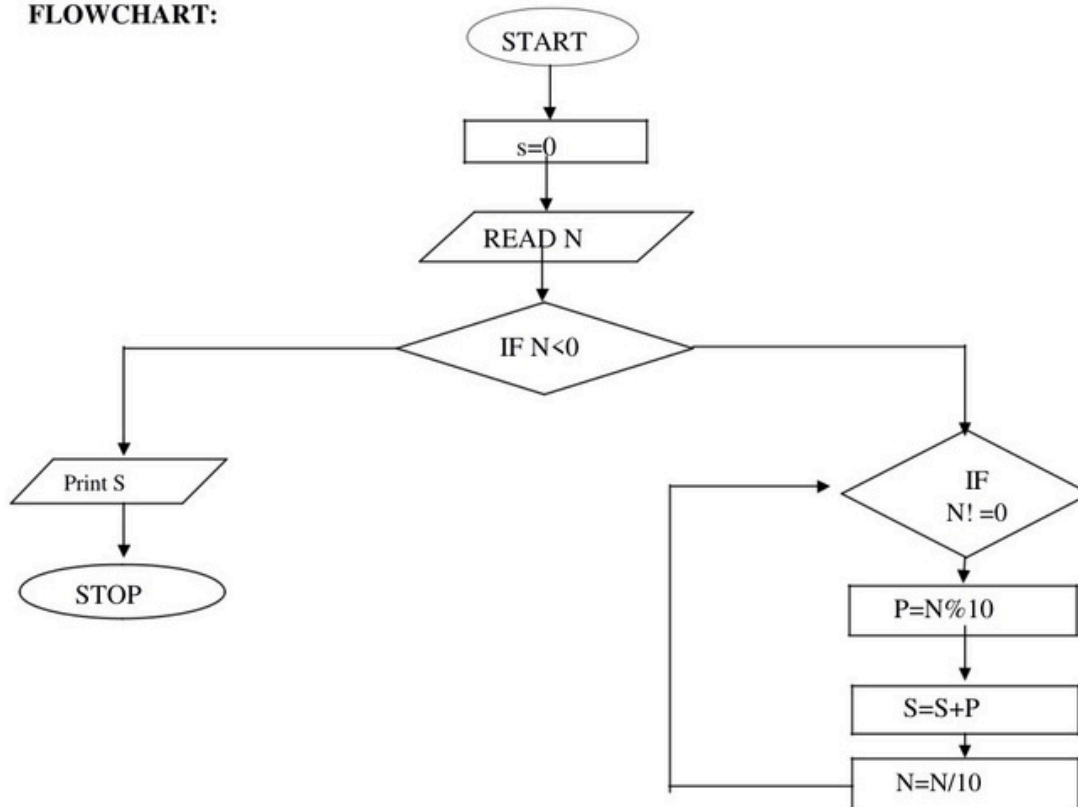
Pre-request/Theme: Students should know about C programming constructs.

Sample program title: SUM OF DIGITS

ALGORITHM:

- Step 1- Open vi editor and start the program
- Step 2- Get number from user. Pass it as argument to function sum.
- Step 3- Find remainder out each individual digit by means of modulo division and calculate sum by adding them.
- Step 4- Display output.
- Step 5- Stop the program.

FLOWCHART:





PROGRAM:

```
#include<stdio.h>
void sum(int);
main()
{
    int no;
    printf("Enter your
    number\n");
    scanf("%d",&no); sum(no);
}

void sum(int num)
{
    int a,sum=0;
    while(num>0)
    {
        a=num%10;
        sum=sum+a;
        num=num/10;
    }
    printf("Sum=%d",sum);
}
```

INPUT	OUTPUT
[com35@localhost ~]\$ gcc sumdigi.c [com35@localhost ~]\$./a.out Enter your number 12345	Sum=15

CONCLUSION:

Thus the program to find the sum of digits was successfully executed and output was obtained.



EX.NO:10

C PROGRAMMING ON UNIX –FILE HANDLING

Date :

Objective: To perform File handling operations using C program on UNIX.

Outcome: Realize how to handle files with C program on UNIX.

Pre-request/Theme: Students should know about C programming constructs.

Description:

Many applications require information stored on auxiliary storage device. Such information is stored permanently as a data file that allows accessing and altering the information whenever necessary.

Opening a File

Prior to performing any activity of a file, the file should be opened. By opening a file, link between the program and the operating system is established. This link exists by means of a structure named FILE, in header file *stdio.h*. Therefore, it is mandatory for programs pertaining to file should include `<stdio.h>`. When a request is made for a file to be opened, the operating system returns a pointer to the structure FILE. The pointer is declared as FILE **fileptr*;

fopen():

A file is opened using the standard function `fopen()`. The file to be opened is searched in the disk. If a file could not be opened, a NULL is returned. `fileptr = fopen("filename", "mode");`
The file mode specifies the purpose of opening a file. Some of them are

Mode Description

- r - Open an existing file for reading only
- w - Open a new file for writing only. If the file exists, its contents are destroyed
- a - Open an existing file for appending. If the file doesn't exist, a new file is created

fread() and fwrite() :

The functions `fread/fwrite` is used for reading/writing data from/to the file opened by `fopen` function. These functions accept three arguments. The first argument is a pointer to buffer used for reading/writing the data. The data read/written is in the form of 'nmemb' elements each 'size' bytes long.

In case of success, `fread/fwrite` return the number of bytes actually read/written from/to the stream opened by `fopen` function. In case of failure, a lesser number of bytes (then requested to read/write) is returned.

fseek():

The `fseek()` function is used to set the file position indicator for the stream to a new position. This function accepts three arguments. The first argument is the FILE stream pointer returned by the



`fopen()` function. The second argument 'offset' tells the amount of bytes to seek. The third argument 'whence' tells from where seek of 'offset' number of bytes is to be done.

`fclose()`:

`int fclose(FILE *fp);`

The `fclose()` function first flushes the stream opened by `fopen()` and then closes the underlying descriptor. Upon successful completion this function returns 0 else end of file (eof) is returned. In case of failure, if the stream is accessed further then the behavior remains undefined. The I/O operation is done using any of the following functions.

`getc & putc`

The `getc` function is used to read a character from the file opened in read mode and assigns it to a character variable. The `getc` will return an EOF marker when the end-of-file is reached. Thereafter reading is not possible. The `putc` function writes the character contained in the variable onto a file opened in write mode. The file pointer moves by one character position for every character I/O operation. `charvar = getc(fileptr); putc(charvar, fileptr);`

`fgets & fputs`

Rather than single characters, to deal with strings `fgets` and `fputs` are used in a similar manner.

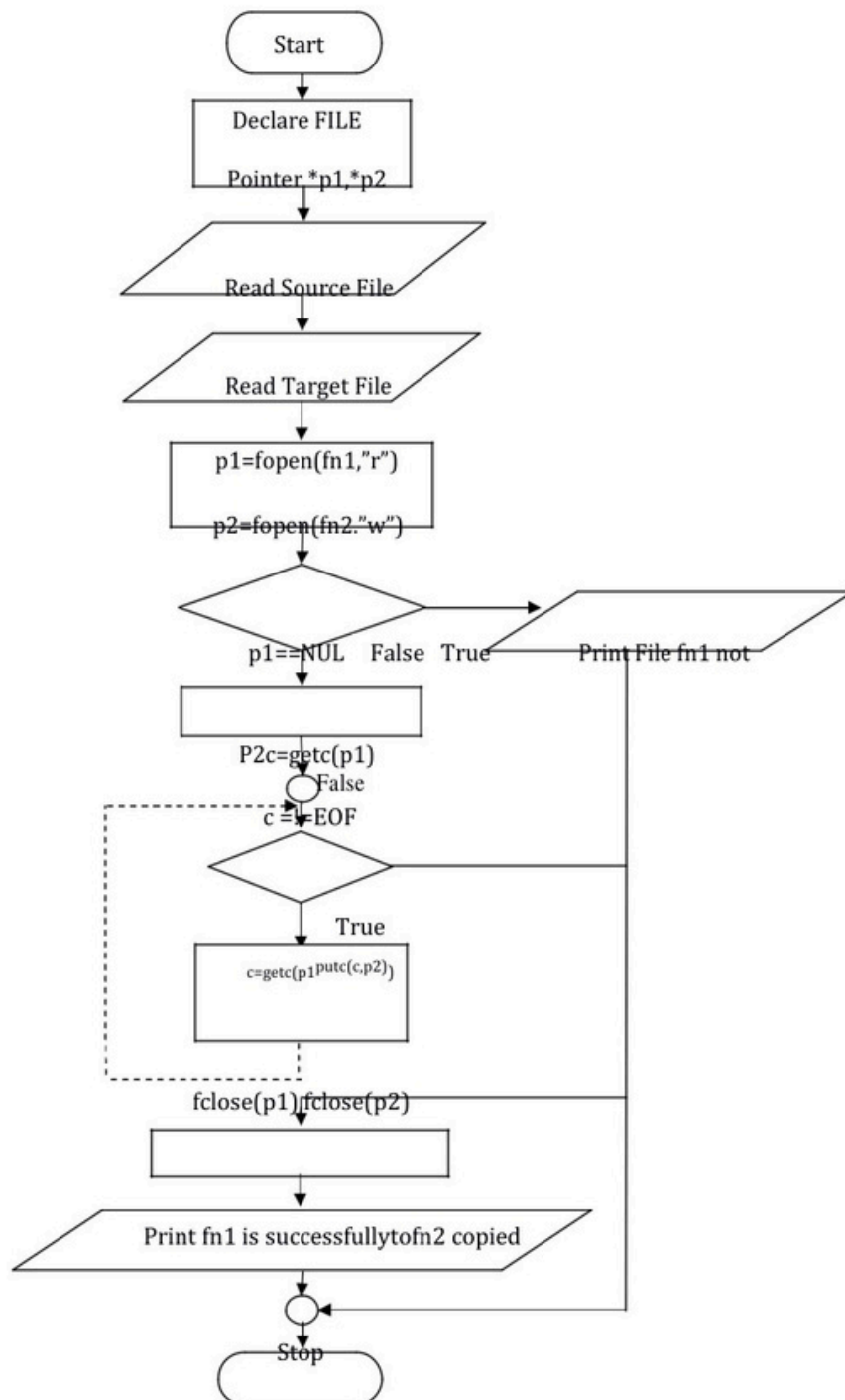
Sample program title: COPY THE CONTENTS OF ONE FILE TO ANOTHER FILE

ALGORITHM:

- Step 1- Open vi editor and start the program
- Step 2- Declare two file Pointer variable `p1` and `*p2`
- Step 3- Read the source filename `fn1`
- Step 4- Read the target filename `fn2`
- Step 5- Assign `p1=open` the file `fn1` in read mode
- Step 6- Assign `p2=open` the file `fn1` in write mode
- Step 7- Test the condition if `p1` is equal to `NULL`, true
 - Print file `fn1` not exist
- Step 8- If the condition fails
 - Read a character from File `fn1` and assign to `c`
- Step 9- While `c` is not equal to end of file and do
 - Putc on File pointer `*p2` position
 - Read a character from File `fn1` and assign to `c`
- Step 10-Close the file pointer `p1` and `p2`



FLOWCHART:





PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
FILE *p1,*p2;
int c;
char fn1[40],fn2[40];
printf("\n Enter the source file ");
scanf("%s",fn1);
printf("\n Enter the Target file");
scanf("%s",fn2);
p1=fopen(fn1,"r");
p2=fopen(fn2,"w");
if(p1==NULL)
{
printf("File %s not exist",fn1);
}
else
{
c=getc(p1);
while(c!=EOF)
{
putc(c,p2);
c=getc(p1);
}
fclose(p1);
fclose(p2);
printf("file %s is successfully copied to file %s",fn1,fn2);
}
}
```

INPUT	OUTPUT
[com35@localhost ~]\$ cat f1 Varuvan vadivelan Institute of Technology Gundalpatti [com35@localhost ~]\$ cc exf1.c [com35@localhost ~]\$./a.out Enter the source file f1 Enter the Target file f2	file f1 is successfully copied to file f2 [com35@localhost ~]\$ cat f2 Varuvan vadivelan Institute of Technology Gundalpatti

CONCLUSION:

Thus the program to copy the contents of one file to another file was successfully executed and output was obtained.